

Playwright + Java + TestNG + POM

Beginner Interview Questions & Answers — Topics 1 to 6

1. Playwright Basics

Q: What is Playwright?

A: Playwright is an end-to-end web automation and testing framework. It supports Chromium, Firefox, and WebKit and can be used with Java.

Q: What are Browser, BrowserContext, Page, and Locator?

A: Browser represents the browser process. BrowserContext is an isolated browser session. Page represents a browser tab/page. Locator identifies an element for interaction or assertions.

Q: What is BrowserContext?

A: It provides an isolated browser session with separate cookies, storage, and authentication state.

Q: What is Page?

A: A Page represents a browser tab or page. Navigation and most UI interactions are performed through a Page object.

Q: What is a Locator?

A: A Locator identifies an element on the page and provides actions such as click(), fill(), and press().

Q: What is headless mode?

A: Headless mode runs the browser without displaying its UI. setHeadless(false) runs it visibly, which is useful for development and debugging.

Q: Explain the Playwright hierarchy.

A: A simple hierarchy is Playwright → Browser → BrowserContext → Page → Locator.

2. Playwright Codegen

Q: What is Playwright Codegen?

A: Codegen records interactions with a web application and generates Playwright code and locators.

Q: Why is Codegen useful?

A: It provides a quick starting point for automation and helps beginners discover possible locators and actions.

Q: Should Codegen output always be used as-is?

A: No. Treat it as a starting point. Review and simplify locators, remove unnecessary actions, and refactor the flow into Page Object classes.

Q: Why can Codegen generate unstable or strange locators?

A: Dynamic text, icons, duplicate accessible names, and the page's DOM structure can cause generated locators to be less stable or readable.

Q: What should you do if a locator matches multiple elements?

A: Make it more specific using a stable attribute, role/name combination, CSS selector, filtering, or another reliable locator strategy.

3. Locators

Q: What is page.locator()?

A: It creates a Locator using a selector such as CSS. For example, locator("#username") targets an element with id=username.

Q: What is getByRole()?

A: It locates an element by its accessible role, such as BUTTON, LINK, or TEXTBOX, and can also use the accessible name.

Q: What is `getByText()`?

A: It locates an element using its visible text, with exact matching available when needed.

Q: What locator strategies do you know?

A: Common strategies are `locator()`, `getByRole()`, `getByText()`, `getByLabel()`, `getByPlaceholder()`, and `getByTestId()`.

Q: Which locator is preferable?

A: Prefer readable, stable locators. User-facing locators such as role and label are often good choices, while stable application-specific attributes are also excellent.

Q: What is strict mode?

A: For actions expected on one element, Playwright can require the locator to resolve to a single element. If multiple elements match, a strict mode violation can occur.

Q: What does `first()` do?

A: It selects the first element matching a locator.

4. Waits

Q: Does Playwright automatically wait?

A: Yes. Playwright provides auto-waiting for many locator actions and waits for elements to become actionable.

Q: What is `waitFor()`?

A: It explicitly waits for a locator to reach the required state before continuing.

Q: What is `waitForURL()`?

A: It waits until the page URL satisfies the supplied URL or pattern condition.

Q: What is `waitForPopup()`?

A: It waits for a new Page opened as the result of an action, such as clicking a link that opens a new tab.

Q: Why do you use `waitForPopup()` in the Patient flow?

A: The Patient Details page opens in a new tab, so `waitForPopup()` captures that new Page object for further interaction.

Q: Why avoid unnecessary `waitForTimeout()`?

A: It is a fixed delay, which can make tests slow and still unreliable. Prefer auto-waiting and condition-based waits where possible.

5. TestNG

Q: What is TestNG?

A: TestNG is a Java testing framework used to organize and execute tests. It provides annotations, assertions, `DataProvider`, grouping, and other test features.

Q: What does `@Test` do?

A: It marks a method as a TestNG test method.

Q: What does `@BeforeMethod` do?

A: It runs before each `@Test` method and is commonly used for setup such as browser initialization.

Q: What does `@AfterMethod` do?

A: It runs after each `@Test` method and is commonly used for cleanup such as closing the browser context.

Q: Why put setup and teardown in `BaseTest`?

A: Common setup and teardown can be reused by multiple test classes, avoiding duplication.

Q: What happens when `PatientTest` extends `BaseTest`?

A: PatientTest inherits the BaseTest setup and teardown methods, so TestNG can run them around the test.

Q: What is an assertion?

A: An assertion verifies an expected condition. For example, `Assert.assertTrue(page.url().contains("/patientsearch"))` verifies that the expected page was reached.

Q: Why use `alwaysRun=true` on `@AfterMethod`?

A: It tells TestNG to run the cleanup method even when the test or configuration encounters a failure.

6. Page Object Model (POM)

Q: What is Page Object Model?

A: POM is a design pattern where page-specific locators and actions are organized into classes representing pages or components.

Q: Why use POM?

A: It separates test flow from page implementation, improving readability, reuse, and maintainability.

Q: What belongs in LoginPage?

A: LoginPage can contain actions such as `openApplication()`, `enterUsername()`, `enterPassword()`, `clickLogin()`, and `login()`.

Q: What belongs in PatientPage?

A: PatientPage can contain actions such as opening the sidebar, opening Patient/Patients, entering a patient ID, searching, and opening patient details.

Q: What belongs in LogoutPage?

A: LogoutPage can contain actions for opening the profile menu, signing out, and verifying that the application returns to the login page.

Q: Why does PatientTest extend BaseTest?

A: BaseTest provides reusable test infrastructure such as Playwright, Browser, BrowserContext, and Page. Page classes provide application-specific behavior.

Q: What is BasePage?

A: BasePage can hold the shared Page reference and common reusable page-level methods for page objects.

Q: How does POM improve your Patient test?

A: The test calls meaningful methods such as `loginPage.login()`, `patientPage.searchPatient()`, `patientPage.openPatient()`, and `logoutPage.performLogout()` instead of containing every UI locator.

Q: How would you explain your framework in an interview?

A: I use Java with Playwright and TestNG. BaseTest manages browser setup and teardown. LoginPage, PatientPage, and LogoutPage contain page-specific actions and locators. Test classes contain the business flow and assertions. I use Codegen as a starting point and refactor the generated code into POM.

Quick Revision

Concept	Remember
Playwright	Web automation/testing framework
Browser	Browser process
BrowserContext	Isolated browser session
Page	Browser tab/page
Locator	Identifies an element
Codegen	Records actions and generates starting code
<code>getByRole()</code>	Find by accessible role/name

Concept	Remember
waitForURL()	Wait for URL condition
waitForPopup()	Capture a newly opened tab/window
@Test	TestNG test method
@BeforeMethod	Runs before each test method
@AfterMethod	Runs after each test method
POM	Separates page actions/locators from test flow
BaseTest	Reusable test setup/teardown